

Delta Lake: Руководство для разработчиков

Middle Level

Практическое руководство по работе с Delta Lake на Rust. Архитектура, протокол, операции и лучшие практики.

1. Архитектура Delta Lake

Delta Lake представляет собой открытый формат хранения данных, который добавляет транзакционные гарантии (ACID) поверх существующих систем хранения. В отличие от обычных файлов Parquet, Delta Lake ведёт журнал транзакций (transaction log), который фиксирует каждое изменение таблицы. Это позволяет реализовать безопасное одновременное чтение и запись, возврат к предыдущим версиям данных (time travel) и аудит изменений. Формат был разработан компанией Databricks и сейчас является open-source проектом Linux Foundation. Библиотека delta-rs предоставляет нативную реализацию на Rust, которая не требует JVM и работает значительно быстрее оригинальной Spark-based реализации.

Основные компоненты архитектуры включают: каталог `_delta_log/` с JSON-файлами коммитов, файлы данных в формате Parquet внутри партиционных директорий, чекпоинты в формате Parquet для ускорения загрузки, и файл `_last_checkpoint` с метаданными последнего чекпоинта. Каждый коммит содержит набор действий (actions): Protocol, Metadata, Add, Remove, CommitInfo и другие. Библиотека delta-rs реализует полный стек протокола, включая чтение лога, управление снапшотами и выполнение операций над таблицей.

2. Настройка проекта на Rust

Для начала работы с delta-rs необходим Rust версии 1.91.1 или выше (edition 2024). Библиотека полностью асинхронная и использует tokio как runtime. Рекомендуемый подход для инициализации проекта выглядит следующим образом:

```
# Cargo.toml
[dependencies]
deltalake = { version = "0.32", features = ["datafusion", "rustls"] }
tokio = { version = "1", features = ["rt-multi-thread", "macros"] }
arrow = "58"

// main.rs
use deltalake::open_table;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let table = open_table("./data/events")?;
    println!("Table version: {}", table.version());
    println!("Files: {:?}", table.get_file_uris()?);
    Ok(())
}
```

Ключевые feature-флаги: `datafusion` включает SQL-движок и возможность выполнять запросы через DataFrame API; `rustls` (по умолчанию) или `native-tls` для TLS-соединений при работе с облачным хранилищем; `s3`, `azure`, `gcp` для поддержки соответствующих облачных хранилищ. Используйте только те features, которые реально нужны, чтобы ускорить компиляцию и уменьшить размер бинарника.

3. CRUD операции с таблицами

Библиотека delta-rs предоставляет полный набор операций для работы с таблицами. Все методы вызываются непосредственно на объекте `DeltaTable` (старый обёрточный API `DeltaOps` объявлен устаревшим). Рассмотрим основные операции с примерами кода.

3.1 Создание таблицы

Таблица создаётся через builder-pattern. Вы определяете колонки (`StructField`), партиционные колонки и имя таблицы. Библиотека автоматически создаёт директорию `_delta_log/` и записывает первый коммит с действиями Protocol и Metadata:

```
use deltalake::operations::create::CreateBuilder;
use deltalake::kernel::{StructField, DataType, PrimitiveType};

let table = DeltaTable::new_in_memory()
    .create()
    .with_table_name("sensors")
    .with_columns(vec![
```

```

StructField::new("id".into(), DataType::Primitive(PrimitiveType::Integer), false),
StructField::new("value".into(), DataType::Primitive(PrimitiveType::Double), true),
StructField::new("ts".into(), DataType::Primitive(PrimitiveType::Timestamp), true),
])
.with_partition_columns(["year", "month"])
.await?;

```

3.2 Запись данных

Данные записываются в формате Arrow RecordBatch. Вы можете выбрать режим сохранения: Append (добавление), Overwrite (полная перезапись) или MergeSchema (слияние схем). Библиотека автоматически создаёт файлы Parquet, вычисляет статистику по колонкам и записывает новый коммит в transaction log:

```

use deltalake::protocol::SaveMode;

let schema = Arc::new(arrow::datatypes::Schema::new(vec![...]));
let batch = RecordBatch::try_new(schema, vec![...]);

table.write(vec![batch])
  .with_save_mode(SaveMode::Append)
  .await?;
// или SaveMode::Overwrite для полной перезаписи

```

3.3 Чтение данных

Чтение выполняется через метод `scan_table()`, который возвращает поток RecordBatch. При этом библиотека загружает снапшот таблицы, получает список активных файлов, отфильтровывает удалённые (Remove actions), и парсит только нужные партиции используя pushdown предикатов. Можно также использовать DataFusion SQL:

```

// Scan API
let (_table, stream) = table.scan_table().await?;
let batches: Vec<RecordBatch> = collect_sendable_stream(stream).await?;

// Или SQL через DataFusion
let ctx = SessionContext::new();
ctx.register_table("sensors", table.into());
let df = ctx.sql("SELECT * FROM sensors WHERE value > 50").await?;

```

3.4 MERGE (Upsert)

Операция MERGE позволяет обновлять, удалять и вставлять строки на основе условий соответствия. Это ключевая операция для ETL и CDC пайплайнов. Синтаксис использует builder-pattern с методами `when_matched_update`, `when_matched_delete`, `when_not_matched_insert`:

```

table.merge(source_df, "target.id = source.id")
  .when_matched_update(|update| {
    update.update("value", "source.value")
    .update("ts", "source.ts")
  })
  .when_not_matched_insert(|insert| {
    insert.all()
  })
  .await?;

```

4. Time Travel и версионирование

Каждый коммит в Delta Lake получает уникальный номер версии. Вы можете загрузить таблицу в состоянии любой предыдущей версии или по временной метке. Это полезно для отладки, воспроизведения результатов, аудита и отката ошибочных изменений. Библиотека поддерживает три режима загрузки: Newest (по умолчанию), Version(n) и Timestamp(dt). При загрузке по временной метке используется бинарный поиск по коммитам для быстрого нахождения нужной версии.

Метод	Описание	Пример
<code>load_version(n)</code>	Загрузка конкретной версии	<code>table.load_version(5).await</code>
<code>load_with_datetime(dt)</code>	Загрузка по timestamp	<code>table.load_with_datetime(dt).await</code>
<code>history(n)</code>	История коммитов	<code>table.history(10)</code>
<code>DeltaTableBuilder.with_version(n)</code>	Версия при открытии	<code>builder.with_version(5)</code>
<code>DeltaTableBuilder.with_datestring(s)</code>	Timestamp при открытии	<code>builder.with_datestring("2025-01-01")</code>

Метод `history(n)` возвращает список `CommitInfo` с метаданными каждого коммита: временем, операцией, пользователем и метриками. Это удобно для аудита и мониторинга изменений в таблице.

5. Оптимизация таблиц

По мере выполнения операций `MERGE`, `UPDATE` и `DELETE` в таблице накапливаются множество мелких файлов и файлы-гумбстоуны. Это ухудшает производительность чтения. Delta Lake предоставляет три ключевые операции для оптимизации.

5.1 OPTIMIZE (Compaction)

Операция `compaction` объединяет мелкие файлы в более крупные, обычно размером от 128 MB до 1 GB. Это значительно сокращает количество файлов и ускоряет последующее чтение. После `compaction` старые файлы помечаются как удалённые (Remove action) и физически удаляются операцией `VACUUM`:

```
table.optimize().with_compaction().await?;
```

5.2 Z-ORDER

`Z-ORDER` группирует данные по указанным колонкам внутри файлов. Это позволяет использовать `min/max` статистику файлов для эффективного пропуска ненужных файлов при фильтрации (data skipping). Эффект особенно заметен при частых запросах с фильтрами по одним и тем же колонкам:

```
table.optimize().with_z_order(["event_type", "region"]).await?;
```

5.3 VACUUM

`VACUUM` физически удаляет файлы, которые больше не используются текущим снапшотом. По умолчанию удаляются файлы старше 7 дней (retention period). Будьте осторожны: после `VACUUM` вы не сможете выполнить `time travel` к версиям, которые ссылались на удалённые файлы. Рекомендуется сначала убедиться, что старые версии не нужны:

```
table.vacuum().with_dry_run(true).await?; // проверка без удаления
table.vacuum().with_retention_period(Duration::from_hours(168)).await?;
```

6. Эволюция схемы

Delta Lake поддерживает изменение схемы таблицы без необходимости переписывать все данные. Вы можете добавлять новые колонки, изменять типы данных (с определёнными ограничениями) и переименовывать колонки. При чтении старых файлов библиотека автоматически подставляет `null`-значения для новых колонок, которых не было в старых файлах. Это позволяет эволюционировать схему постепенно, без даунтайма.

```
table.add_constraint().with_constraint("id > 0").await?;
table.add_feature().with_feature("deletionVectors").await?;
```

7. Работа с облачными хранилищами

Библиотека `delta-rs` поддерживает три основных облачных хранилища: Amazon S3, Azure Blob Storage и Google Cloud Storage. Конфигурация передаётся через `HashMap<String, String>` при создании таблицы. Для S3 необходимо указать `credentials` и `region`. Для Azure - `connection string` или `SAS token`. Для GCP - `service account key` или `IAM credentials`.

Хранилище	Feature flag	Ключевые параметры
Amazon S3	s3	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
Azure Blob	azure	AZURE_STORAGE_ACCOUNT_NAME, AZURE_STORAGE_ACCOUNT_KEY
Google GCS	gcp	GOOGLE_SERVICE_ACCOUNT, GOOGLE_PROJECT_ID
Local FS	(default)	Просто путь к директории

Для S3 также поддерживается `commit-through-DynamoDB` для сильной согласованности. Это особенно важно при одновременной записи в таблицу из нескольких процессов. Без `DynamoDB` используется оптимистичная блокировка через `try-retry` механизм.

8. Лучшие практики

- Размер файлов: Стремитесь к размеру файлов 256 MB - 1 GB. Мелкие файлы ухудшают производительность, слишком большие - усложняют чтение с фильтрами.
- Партиционирование: Партиционируйте по колонкам с низкой кардинальностью (дата, регион). Слишком много партиций создаёт проблему `small files`.

3. Регулярная оптимизация: Запускайте OPTIMIZE после каждых 5-10 MERGE/UPDATE операций для поддержания производительности чтения.
4. VACUUM с осторожностью: Не уменьшайте retention period ниже 7 дней. Убедитесь, что time travel к старым версиям больше не нужен.
5. without_files(): Для append-only приложений используйте .without_files() в DeltaTableBuilder. Это значительно экономит память.
6. Log buffer size: Настройте буфер лога под ваши хранилища. Для облачных хранилищ с rate limits уменьшите значение по умолчанию (4 x CPU cores).
7. Не используйте DeltaOps: Старый API DeltaOps(table) устарел. Используйте методы напрямую на DeltaTable.
8. Feature flags: Включайте только необходимые флаги. Каждый дополнительный feature увеличивает время компиляции и размер бинарника.